



Universidade Federal do Rio Grande do Norte

Alice in the Uberland

Ian Gabriel, Lucas Apolônio, Eduardo Marinho

2026-09-09

1 Grafos1

2 Strings1

Grafos (1)

1.1 Matching

MaximumMatching.cpp

Description: Matching máximo de um grafo bipartido. n e m são os tamanhos das partes. $mt[i]$ é vértice da parte A que deu match com i (i é da parte B).

Time: $\mathcal{O}(VE)$ fa10ea, 24 lines

```
vector<int> g[MAXA];
bool used[MAXA];
int mt[MAXB], n, m;

bool findPath(int x) {
    if(used[x]) return false;
    used[x] = true;
    for(int y : g[x]) {
        if(mt[y] == -1 || findPath(mt[y])) {
            mt[y] = x;
            return true;
        }
    }
    return false;
}

int maximumMatching() {
    for(int i=0; i<m; i++) mt[i] = -1;
    int ans = 0;
    for(int i=0; i<n; i++) {
        for(int i=0; i<n; i++) used[i] = false;
        ans += findPath(i);
    }
    return ans;
}
```

Strings (2)

2.1 Automatos

AhoCorasick.cpp

Description: Autômato de Aho-Corasick. $lnk[i]$ é o suffix link de i . $go[i][j]$ é o automato. $ex[i]$ é o proximo vertice terminal seguindo pelos links de i , e $nmatches[i]$ é a quantidade de matches do estado i .

Usage: start no começo, adiciona as strings, depois rode o build

Time: Inserção $\mathcal{O}(|S|)$. Build $\mathcal{O}(\sum |S_i|k)$ 81e8ef, 50 lines

```
namespace Aho {
    int lnk[MAX], go[MAX][K], ex[MAX], nmatches[MAX];
    bool output[MAX];
    int tam;

    void start() {
        tam = 1;
        memset(go, -1, sizeof(go));
        memset(lnk, -1, sizeof(lnk));
        memset(ex, -1, sizeof(ex));
    }

    void add_string(string &t) {
        int r = 0;
        for(char ch : t) {
            int c = valor(ch);
```

```
            if(go[r][c] == -1) {
                go[r][c] = tam; tam++;
            }
            r = go[r][c];
        }
        output[r] = nmatches[r] = true;
    }

    void build() {
        queue<int> q;
        lnk[0] = ex[0] = 0;
        for(int i=0; i<K; i++) {
            if(go[0][i] == -1) go[0][i] = 0;
            else {
                lnk[go[0][i]] = ex[go[0][i]] = 0;
                q.push(go[0][i]);
            }
        }
        for(;!q.empty(); q.pop()) {
            int v = q.front();
            int u = lnk[v];
            for(int i=0; i<K; i++) {
                if(go[v][i] == -1) go[v][i] = go[u][i];
                else {
                    lnk[go[v][i]] = go[u][i];
                    int w = lnk[go[v][i]];
                    ex[go[v][i]] = (output[w] ? w : ex[w]);
                    nmatches[go[v][i]] += nmatches[w];
                    q.push(go[v][i]);
                }
            }
        }
    }
}
```